HACKATHON / 48-HOUR BUILD

LabLoop

Unlock underused equipment and spaces.

Trusted campus sharing, from eligibility and human approval to condition evidence and return.

PROVISIONAL - NOT SUBMISSION READY

Local product independently verified. Public hosting, the official PDF template and verified team details remain pending.

Selected use case

Campus resource sharing at fictional Northstar Institute of Applied Technology

Runtime

Next.js 16 + React 19 + TypeScript + persistent local SQLite

Official deadline

1 November 2026, 10:00 AM IST

Source repository

https://github.com/Dhaksith-S/labloop-hackathon

Team: [TO BE CONFIRMED] / Participant IDs and actual contributions: pending

Document status: Provisional content and PDF. The official PDF template is unavailable; this document does not claim format compliance. Team identities, participant IDs and real contributions await confirmation. Nothing has been submitted automatically.

Authority: The organiser brief and 100-point rubric were supplied by the project owner and transcribed in docs/OFFICIAL_PROBLEM_STATEMENT.md. The campus use case is the selected scope. The deadline is **1 November 2026, 10:00 AM IST**.

Challenge: Unlock Underused Equipment and Spaces

Product: LabLoop - a trusted campus resource-sharing and custody loop

Build date: 30 October 2026

Status: Locally runnable and tested. **Not publicly deployed.** Supabase schema/RLS prepared but unapplied. No invented adoption, usage or deployment claims.

Source: https://github.com/Dhaksith-S/labloop-hackathon (public repository; no hosted application).

1. Executive summary

Campuses already own the equipment students need. The bottleneck is not inventory; it is trusted access. Informal sharing fails because nobody can answer three questions quickly:

1. Is this person allowed to use this machine?
2. Is the slot actually free?
3. What condition was it in when it left, and who had custody?

LabLoop answers all three in one lifecycle: a verified owner publishes rules and hours, a student sees exactly which rules they meet, a human owner decides, the borrower checks in with condition evidence, returns it with final evidence, and the owner closes custody. Conflicts are blocked at both the service and database layers. Rejections and waitlists require written reasons. A released slot reopens the first eligible waitlist request for human review - never automatic approval.

The product is a complete Next.js monolith with a persistent database, demo institution, role-aware UI, audit trail, incidents and credential verification. The original verification passed 44 unit/integration tests and 5 Chromium browser tests. Subsequent Funnel preparation added four ingress tests (48 at that stage), and Cloudflare preparation added three more (51 total). The production build was retained and the isolated demo lifecycle verified locally. An approved Funnel attempt failed public reachability and was disabled. The mandatory working public application requirement remains unsatisfied.

2. The problem, concretely

A lab steward fields requests in chat. A student asks for a printer at 2 pm; the steward cannot remember whether the safety induction happened, whether another print is running, or whether the laser is down for servicing. A second student is told "maybe" and never hears back. When a probe disappears, nobody can prove who had the kit.

The brief asks for underused equipment and spaces to be unlocked. The hard part is not a calendar. It is **eligibility plus accountability**.

Target users and selected use case

User	Need	LabLoop response
Students and researchers	Find equipment they cannot individually afford; understand access conditions	Public discovery, requirements, explained eligibility, booking and custody timeline
Verified resource owners / lab stewards	Share underused assets while retaining control	Institution-assigned owner role, own-resource listings, written human decisions, return confirmation
Lab managers / administrators	Verify credentials and oversee safety and allocation	Organisation-wide review, credential verification, incidents and audit history
Judges / new demo users	Independently exercise a complete journey	Fictional accounts, visible demo picker, reproducible setup and step-by-step walkthrough

The focused demonstration is one fictional campus. The wider brief also covers communities and small businesses, but LabLoop does not claim a multi-community marketplace, payments, seat inventory or real institutional integration.

Problem evidence and assumptions

The organiser-provided brief identifies the access and trust problem. The lab scenario above is a reasoned illustration, not an interview or measured study. No utilisation baseline, savings estimate, demographic finding, market survey or adoption metric was collected. Future validation should interview borrowers and stewards, test whether requirements are understandable, and measure completed loans and owner workload with consent before making impact claims.

Complete transaction workflow

1. An institution assigns an owner/manager role; student signup never grants staff authority.
2. The owner publishes a resource with rules, condition, location, hours, caps and optional required credentials.
3. A student searches/filter the public catalog and checks the listed requirements.
4. The request form checks active membership, human-verified credentials, duration, hours, blackouts, conflicts and fair use.
5. A passing request enters pending; the owner or authorised manager reviews it.
6. Approval rechecks eligibility; rejection/waitlisting records a written reason. Conflicting approved slots are blocked in SQL.
7. The approved borrower acknowledges rules and checks in inside the permitted window, appending initial condition/accessory evidence.
8. The borrower submits final condition and return notes. The owner independently confirms the hand-back.
9. Damage, delay or access conflicts can be reported for human review. Overdue state is derived from time; no automatic penalty is applied.
10. Borrower and authorised staff inspect the timeline, both condition reports and attached incidents. History cannot be edited through application paths.

3. What was built

Public discovery

- Branded landing page explaining the loop without fabricated social proof.
- Searchable, category-filtered catalog of active listings.
- Resource detail: description, location, owner, condition, usage rules,

published weekly hours, blackouts, required credentials and request CTA.

- Light/dark theme, responsive layout, keyboard-visible focus.

Student workspace

- Dashboard with real counts and next actions.
- My bookings with status, next step, waitlist position and overdue indicator.
- Eligibility panel with pass / warning / fail detail before submission.
- Booking detail with status timeline, condition reports, incidents, cancel and check-in/return forms.
- Activity feed, persistent notifications, profile with credentials and history.

Owner studio

- Owner dashboard scoped to owned resources.
- Request queue with approve / reject / waitlist; rejection and waitlisting require written reasons.
- Custody queue and return confirmation.
- Incident queue with human resolution notes and self-review blocking.
- Listing management: create, edit, pause/archive, requirements, weekly availability, blackouts.
- Organization audit activity.

Manager/admin

- Organization overview with SQL-backed counts: listings, custody, members, roles, open incidents, recent audit.
- Organization-wide qualification verification with reviewer and expiry.
- Access to all-org requests, custody and incidents (role-scoped).

Integrity and security

- Server-side authn/authz; scrypt passwords; HMAC-hashed session tokens; HttpOnly SameSite cookies; login throttling; open-redirect protection.
- Deterministic eligibility engine; approval-time re-validation.
- Atomic booking writes; SQLite overlap triggers; prepared PostgreSQL exclusion constraint.
- Append-only audit, timeline and condition records enforced by database triggers.
- Self-approval and self-incident-review rejection.
- Zod at every form boundary; prepared SQL throughout.
- Security headers.

Operations

- Idempotent fictional seed with five accounts, seven resources, qualifications, bookings, incidents, conditions, notifications and audit history.
- Setup, seed and reset scripts.
- Isolated E2E database and server port.

- Prepared Supabase schema + RLS with explicit unverified status.

4. What is genuinely working

The following were executed on 30 October 2026 in this workspace:

Verification	Result
npm test	4 files, 44 tests passed
npm run test:e2e	5 tests passed, Chromium; includes listing/discovery and 390 px mobile checks
npm run typecheck	passed
npm run lint	passed
npm run build	passed, all routes compiled
npm run verify:production	home/login/catalog HTTP 200; headers and Secure cookie; browser custody; session/completed booking/condition records survive local process restart; verified backup
Existing demo database	Preserved; recovery backup made before testing; no reset run in this takeover

The table is the original baseline. Latest checks passed 51 tests, typecheck/lint and the Cloudflare-mode production lifecycle, restart/session and backup restore. The five standard E2E tests were not rerun; public transport remains unverified.

The E2E test completes request -> approve -> check-in -> return -> confirm across student and owner accounts in a browser. It is not a mocked component test.

The integration suite includes a direct SQL insert that bypasses application code and is rejected by the database overlap trigger, and direct UPDATE/DELETE attempts rejected by append-only triggers.

The takeover also tests direct student access to staff queues, multi-organisation owner scoping, suspended staff, token expiry/revocation, production-secret failure, URL redirect normalization, skipped custody transitions, revoked credentials at approval, self-return confirmation and WAL-safe backups. The command/evidence log is docs/VERIFICATION.md; detailed inventory and limitations are in docs/TESTING.md.

5. Design decisions

Monolith over microservices

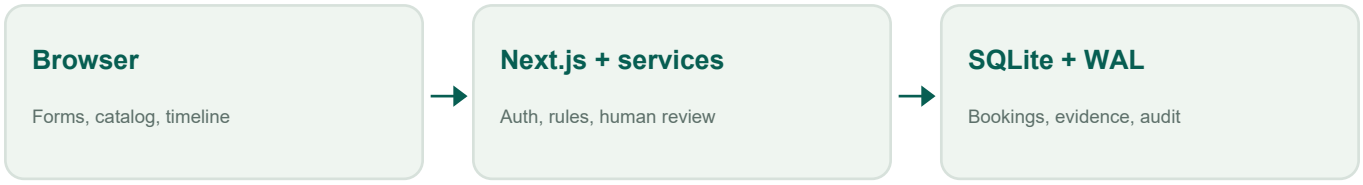
One Next.js app and one relational store fit a 48-hour build and an institution's actual scale. Domain rules are plain functions, callable from pages, actions and tests. There is no network boundary between eligibility and booking creation.

SQLite over an unconfigured cloud

No Supabase credentials existed. Rather than ship a fake backend or an unverified cloud claim, the runnable product uses persistent SQLite. The PostgreSQL/RLS path is included as a prepared migration with its status stated plainly.

This is a deliberate trade-off: excellent local reproducibility, but not serverless-persistent. It is the main deployment blocker.

Architecture and technology stack



Runtime architecture. Supabase SQL is prepared separately and is not connected.

The browser sends forms to Next.js 16.4.3 Server Actions. Actions validate with Zod 4 and resolve the authenticated session; plain TypeScript services enforce membership, role, resource ownership, eligibility and legal state transitions. Services issue bound SQL through better-sqlite3. Booking state, evidence, audit and notifications commit together in immediate SQLite transactions. Database triggers enforce append-only evidence and resource overlap protection.

Layer	Actual technology / data
UI and server rendering	React 19.3, Next.js 16 App Router, TypeScript, Tailwind CSS v4
Interaction	Server Actions, small client components for forms, theme and navigation
Authentication	Node scrypt password hashes, random opaque tokens stored as HMAC hashes, secure production cookies
Persistence	Local SQLite file, WAL, foreign keys, prepared statements, transactions and triggers
Core records	Profiles, memberships, resources, requirements, qualifications, availability, bookings, conditions, incidents, timeline, audit, notifications and sessions
Verification	Vitest 5.0.5, Playwright 1.64.1 Chromium, TypeScript, ESLint, Next production build
Prepared alternative	Supabase/PostgreSQL schema and RLS SQL only; not connected or tested

Timestamps are stored as ISO UTC and availability/display interpreted in Asia/Kolkata. Public catalog reads exclude inactive resources. Private booking pages authorize the borrower, active resource owner or active organisation manager/admin. The complete architecture is in docs/ARCHITECTURE.md.

Human decision over auto-approval

Eligibility is necessary but not sufficient. Owners see context a rule engine cannot: a broken fixture, a student who needs accommodation, a lab tech's judgment. The product automates verification and evidence, not allocation. Rejection and waitlisting require written reasons. Approval cannot waive a required eligibility rule.

Transparency over a hidden score

Waitlist order is submission time, visible to the borrower. There is no reputation score, priority tier or paid fast lane. The trade-off is a queue that can be gamed by early requests, mitigated by per-resource fair-use caps.

Boring, auditable primitives

Server Actions, Zod, prepared SQL, database triggers and append-only tables. No custom crypto, no ORM magic, no client-side authorization.

6. UX and visual direction

The interface uses a "technical institution" visual language: warm paper surfaces, deep green primary, amber accents, Manrope for UI, Newsreader italic for emphasis, JetBrains Mono for labels. Listing art is deterministic CSS/SVG by category rather than stock photography.

The goal is credibility: a lab steward should believe this could replace their spreadsheet. Every status has text, not color alone. Empty states explain what will appear. Forms show field-level errors and pending states. The layout works from mobile to wide desktop, in both themes.

7. Deployment reality

No verified working public URL. Paid resources are prohibited. The selected zero-cost route preserves the Next.js production build and persistent SQLite on the owner's Windows laptop, with a fictional judge database outside OneDrive. The original application database is unchanged. A persistent unique session secret is Windows-DPAPI encrypted; Secure, HttpOnly, SameSite=Lax cookies are verified locally. Backend and filtered ingress bind only to loopback. Exact Host/Origin validation is enforced; administrative and development paths are blocked. No unrestricted Server Actions origin wildcard was introduced.

The configured-origin browser journey passed locally: request, approval, check-in, initial/final reports, owner return confirmation and persisted history. Session and records survived application restart; online backup restoration passed SQLite integrity checking. Local interception is explicitly not public deployment evidence.

After separate user approval and HTTPS/Funnel enablement, Tailscale 1.104.4 accepted HTTPS 443 -> filtered ingress 127.0.0.1:3211 -> production app 127.0.0.1:3210, issuing https://dhaksith.tail3cd298.ts.net/. Public HTTPS and browser catalog probes timed out before login. Funnel was disabled as required; final status is {}. Later DNS/TCP diagnostics succeeded and the local catalog returned HTTP 200, but the original timeout cause remains unresolved. No public lifecycle, certificate validation, independent-device access or URL stability has been verified. Do not submit that disabled address as a working application.

Cloudflare Quick Tunnels is a no-cost, accountless fallback. Approved local preparation verified portable cloudflared 2026.10.6 signature/checksum/version and its ingress mode's production lifecycle, restart/session and backup restore. The user manually allocated a URL and started the approved app after automated launches were blocked. Local catalog returned 200, but public access returned 530/error 1033 because no cloudflared process remained. The process exit cause is unknown. App/ingress were stopped safely for recovery; no public lifecycle passed. Hostnames change on recreation and uptime is not guaranteed. The laptop and network must stay online. No power settings were changed. The paid Render template remains unapplied; Supabase remains an unapplied port. See docs/TAILSCALE_FUNNEL.md for commands, failed-attempt evidence; docs/CLOUDFLARE_QUICK_TUNNEL.md records the local fallback evidence, two approval gates and remaining public acceptance tests.

8. Limitations

- No public deployment; no Vercel/Supabase verification.
- Supabase SQL and RLS are unexecuted and untested.
- Chromium-only E2E; desktop and 390 px viewport, no real mobile device or Firefox/WebKit matrix.
- No automated accessibility audit; no WCAG conformance claim.
- No load or concurrency stress test.
- No email verification, password reset, MFA or OAuth.
- No photo upload or storage bucket, despite photo_url in the model.
- No background scheduler for waitlist expiry or reminders.
- No scheduled overdue notifications: approval/cancellation/custody messages are in-app, while overdue is a derived indicator.
- No appeal/second-review workflow.
- No incident-to-eligibility policy; incidents are recorded but do not

automatically restrict future access.

- Login throttling is in-process only.
- No CSP or HSTS at the app layer.

- Single-capacity resources only; no seat/quantity inventory.
- No multi-institution tenant switcher in the UI (roles and data are organization-scoped, but the demo has one institution).

9. If there were another 48 hours

Priority order:

1. **Authorized persistent hosting.** Apply the prepared single-instance Node configuration only after a compatible persistent host and budget are approved.
2. **Public acceptance.** Verify the full loop, permissions, restart durability, backups, HTTPS cookies and independent judge access at the real URL.
3. **Waitlist operations.** Expiry, reminders, review SLA and aging indicators.
4. **Appeals and second review.** Preserve the first decision, add an independent reviewer.
5. **Photo evidence.** Supabase Storage with signed URLs, type/size limits and storage RLS.
6. **Accessibility verification.** axe/Lighthouse plus keyboard and screen-reader sessions with students.
7. **Notification delivery.** Email or push with preferences, while keeping the in-app record authoritative.
8. **Operational reporting.** Incident consistency review, utilization without fabricated metrics, exportable audit for institutional review.

10. Official judging rubric - evidence, not a predicted score

Official criterion	Points	Actual evidence / gap
Understanding of the Problem	15	Access/trust/accountability scenario and target users; no invented research
Strength of the Core Solution	20	Focused listing-to-custody services, eligibility and human decisions
Working End-to-End Product	20	Local browser listing and complete custody loop; public access still missing
Technical Depth and Reliability	15	Domain services, atomic lifecycle, database guards, 44 tests and production build
Trust, Safety and Accountability	10	Roles, verified credentials, immutable conditions and human incident review; formal disputes absent
Fair Allocation and Accessibility	10	FIFO, fair-use caps, published reasons, mobile/semantic checks; no WCAG certification
Originality and Real-World Potential	10	Eligibility plus custody/evidence differentiates the workflow; adoption unvalidated

docs/JUDGING_TRACEABILITY.md maps all seven criteria and mandatory workflow steps to actual code paths, executed tests and gaps. No marks are self-awarded.

11. Claim boundaries

True:

- The app runs locally with persistent SQLite.
- The listed tests, typecheck, lint and build passed.
- Schema and RLS SQL for Supabase exist and are documented as unverified.
- The demo institution and people are fictional.

Not true / not claimed:

- A live URL or production deployment.
- Supabase applied, RLS verified or storage configured.
- Real institutional adoption, user counts or utilization metrics.
- WCAG compliance, penetration-test clearance or load capacity.
- Any integration with a real campus identity provider.

12. Security, privacy and fairness

Staff authority is checked in services as well as pages. The takeover closed missing student-role guards in staff queues, organisation leakage for owners, suspended-staff read gaps, a browser-normalized redirect edge case and a direct SQL resource-reassignment overlap gap. Check-in also rechecks current database membership, preventing custody after suspension. Regression tests cover these findings. Owners cannot approve their own booking or confirm their own return; reporters cannot review their own incident; managers cannot verify their own qualification.

Profiles contain email/name and password hashes; sessions store hashes only. Condition notes, incident descriptions and decisions can be sensitive even when no photo is uploaded. Public resource pages expose listing information and owner display details, while participant records require authorization. There is no analytics, advertising, external font request or runtime AI service. Databases, backups and secrets must remain private and excluded from Git.

The public seed and accounts are fictional demonstration data. Judges should avoid real personal information and use test credentials. Real institutional onboarding requires identity verification, recovery/MFA strategy, rate limits, HTTPS/security headers, private backups, retention/deletion policy and an operational owner. These are not a security certification or legal compliance claim. SQLite file administrators can still change schema or restore old files.

Fairness is procedural: published requirements, the same resource cap for each member, written reasons, submission-time queue order, eligibility rechecks and human final decisions. Incidents do not impose automatic penalties. There is no demographic scoring, priority purchase or hidden reputation score. Formal appeals, review SLAs, accommodations and cross-resource caps remain future work.

13. Judge instructions and submission readiness

For local review, use Node 24+, run npm install, npm run db:setup, then npm run dev. Setup preserves existing profiles; do not reset a database with judge-created records. Open http://localhost:3000 and the visible demo picker. All fictional demo accounts use LabLoop!2026.

1. As owner@labloop.demo, publish a listing with rules and hours; inspect it in Discover.
2. As priya@labloop.demo, choose Open Maker Bench 04, request a slot starting in about 15 minutes and ending one hour later, enter a purpose and acknowledge rules.
3. As the owner, open the booking and approve after reviewing eligibility.
4. As Priya, check in with initial condition, then submit return with final condition.
5. As the owner, confirm return; refresh and inspect completed status, both condition reports and timeline.
6. For incident handling, use the booking's incident form and the owner's Incident desk. A different authorised reviewer handles the outcome.

7. As manager@labloop.demo, inspect the organisation overview and verify a pending qualification.

The mandatory public application is **pending**. Source is accessible at the existing public GitHub repository. The report PDF is provisional. Official template conversion, verified team/participant details and each member's own submission are pending. The form is https://forms.gle/QTD1ZRuPX37Yef37A; it has not been submitted by Codex.

14. Team and actual personal contributions - pending verification

Team name: [TO BE CONFIRMED BY TEAM]

Participant name	Participant ID	Actual role	Actual personal contribution and evidence
[MEMBER 1 - REAL NAME PENDING]	[ID PENDING]	[ROLE PENDING]	[VERIFIED CONTRIBUTION / COMMITS PENDING]
[MEMBER 2 - REAL NAME PENDING]	[ID PENDING]	[ROLE PENDING]	[VERIFIED CONTRIBUTION / COMMITS PENDING]
[ADD / REMOVE ROWS TO MATCH ACTUAL TEAM]	[ID PENDING]	[ROLE PENDING]	[VERIFIED CONTRIBUTION PENDING]

Demo account names are fictional users, **not team members**. Git authorship alone does not establish an individual's personal contribution. The team must review this table, confirm provenance and prepare individual submissions.

15. Libraries, pre-existing material and AI assistance

The original framework scaffold and third-party packages are allowed external building blocks, not original project inventions. Next.js/React/TypeScript, Tailwind, better-sqlite3, Zod, Lucide, Radix Slot, Fontsource, Vitest, Playwright and supporting libraries are listed with versions/licenses in docs/THIRD_PARTY_DISCLOSURES.md. No stock imagery, tutorial theme, paid asset, runtime AI integration, Supabase service or Vercel deployment is claimed.

DeepSeek assistance: The project owner reports that DeepSeek V4.1 Flash assisted the existing application and documentation before its quota expired. The precise model label and per-file authorship are user-reported, not verified by Codex. Baseline commits, code and tests were inherited and preserved.

Codex assistance: Codex inspected and backed up interrupted work, verified the baseline independently, audited services, implemented the documented security/test-safety fixes, expanded tests, prepared persistent-volume startup and backup tooling, and prepared evidence-based documentation and this provisional PDF. Report generation uses Python ReportLab; PyMuPDF renders pages for visual review. These are development tools, not application integrations.

Provenance and development window: The organiser specifies a 48-hour window from 30 October 2026, 10:00 AM IST to 1 November 2026, 10:00 AM IST. The first five Git commit author timestamps are on 30 October between 13:11 and 13:41 IST. This provides development evidence but does not independently prove the origin of every line. [TEAM TO CONFIRM ANY OTHER PRE-EXISTING CODE/ASSETS AND THEIR LICENCES BEFORE SUBMISSION.] No invented human contribution is assigned.

16. Evidence and references

- Organiser-provided problem statement/rubric, transcribed in docs/OFFICIAL_PROBLEM_STATEMENT.md.
- Existing public source: https://github.com/Dhaksith-S/labloop-hackathon.
- Code/test traceability: docs/JUDGING_TRACEABILITY.md and docs/TESTING.md.
- Independently executed commands, failures/fixes and limits: docs/VERIFICATION.md.
- Architecture, privacy/security and fairness: docs/ARCHITECTURE.md, docs/SECURITY.md, docs/FAIRNESS.md.
- Persistent-store comparison and provider documentation: docs/DEPLOYMENT.md.